

Cómo hacer tareas comunes

Guías paso a paso para las operaciones más frecuentes del sistema. Estas guías asumen que ya completaste el tutorial de inicio.

Cómo preparar la conexión a MySQL

La persistencia del sistema vive en una base de datos MySQL. Antes del primer arranque:

1. Instala el conector: `pip install mysql-connector-python`.
2. Asegúrate de que el servidor MySQL esté en ejecución en `localhost:3306`.
3. Abre `persistencia.py` y edita el bloque `CONFIG_MYSQL` con tus credenciales:

```
CONFIG_MYSQL = {
    "host": "localhost",
    "port": 3306,
    "user": "root",
    "password": "",          # ← pon aquí la contraseña de tu MySQL
}
BASE_DATOS = "crisis_monitor"
```

Al iniciar `main.py`, el sistema ejecuta `CREATE DATABASE IF NOT EXISTS crisis_monitor` y crea las tablas `comentarios` y `alertas` automáticamente. **No necesitas crearlas a mano.**

Creación manual (opcional): si prefieres preparar la base por adelantado, el proyecto incluye `schema.sql`. Ejecútalo con `mysql -u root -p < schema.sql`. El sistema **no** lee ese archivo en el arranque; es solo una conveniencia para crear el esquema desde la línea de comandos.

Cómo cargar tu propio archivo CSV

Prerrequisitos

Tu CSV debe tener al menos estas tres columnas (con estos nombres):

Columna	Tipo	Descripción
id	texto	Identificador único del comentario
texto	texto	Contenido del comentario a analizar
fecha	texto	Fecha del comentario (formato libre, ej: 2026-04-16)

La columna `usuario` es opcional: si está presente, el sistema la conserva y la almacena (base de datos, CSV de exportación y alertas), aunque no influye en la puntuación de sentimiento. Cualquier otra columna adicional se ignora. La codificación preferida es UTF-8; si el archivo está en latin-1, el sistema reintenta automáticamente.

Pasos

1. Coloca tu CSV en el directorio del proyecto (o anota la ruta completa).
2. Ejecuta el sistema: `python main.py`
3. Elige la opción `1` en el menú.
4. Escribe la ruta de tu archivo cuando se solicite:

```
Ruta del CSV (Enter = 'comentarios_ejemplo.csv'): mis_comentarios.csv
```

Verificación

Si la carga es exitosa verás el resumen de etapas (leídos / únicos / ruido / duplicados / clasificados) y la línea de persistencia `MySQL → M filas | K alertas`. Si el archivo no existe o faltan columnas, el sistema lo informa en consola sin abortar.

Cómo ajustar el umbral de alertas

El umbral controla cuán negativo debe ser un comentario para disparar una alerta individual. El valor por defecto es `-2.0`. Un umbral de `-1.0` es más sensible (más alertas); `-4.0` solo alerta los casos extremos.

1. En el menú principal, elige la opción **8**.
2. Verás el umbral actual y la escala de uso.
3. Escribe el nuevo valor (ej: **-1.5**).
4. Carga un CSV nuevamente (opción **1**) para que el nuevo umbral se aplique al análisis.

Si ingresas un umbral positivo, el sistema pedirá confirmación porque eso significa que ningún comentario negativo disparará alerta individual.

Cómo exportar datos en distintos formatos

Prerrequisito: debes haber cargado un CSV en la sesión actual (opción **1**).

1. Elige la opción **6** en el menú principal.
2. Selecciona el formato:

Opción	Formato	Contenido
1	TXT	Todos los comentarios con categoría y palabras clave
2	CSV	Solo comentarios negativos y muy negativos
3	Pickle	Backup completo: datos + alertas (para restaurar en Python)
4	Todos	Los tres formatos anteriores a la vez

Los archivos de exportación llevan sello de tiempo (`YYYYMMDD_HHMMSS`) para que múltiples exportaciones no se sobrescriban.

Cargar un Pickle en Python

```
import pickle
with open("datos_salida/datos.pkl", "rb") as f:
    contenido = pickle.load(f)
datos = contenido["datos"] # list[dict] - comentarios analizados
alertas = contenido["alertas"] # list[dict] - alertas generadas
```

Cómo consultar y gestionar la base de datos

Consultas de solo lectura (opción 3)

Sub-opción	Qué muestra
1	Últimos 20 comentarios ordenados por fecha de proceso
2	Comentarios filtrados por categoría (elegir de lista)
3	Solo las alertas activas (no resueltas)
4	Todas las alertas (activas y resueltas), máx. 20

Resolver una alerta (opción 4 → 2)

Cuando el equipo atiende una crisis, puedes marcar la alerta como resuelta (operación **UPDATE** sobre la tabla `alertas`): usa la opción **3 → 3** para ver las alertas activas y anotar el ID, y luego la opción **4 → 2** para escribir ese ID. El sistema responde `Filas actualizadas: 1`.

Limpiar alertas resueltas (opción 4 → 3)

Para eliminar definitivamente las alertas ya resueltas (operación **DELETE**), el sistema reporta cuántas filas eliminó.

Atención: esta operación es irreversible. Las alertas eliminadas no se pueden recuperar.

Cómo regenerar el dataset desde texto crudo (limpiar.py)

El script `limpiar.py` es la utilidad con la que el equipo construyó `comentarios_x.csv` a partir de comentarios públicos recopilados manualmente. La recolección automática desde X queda fuera del alcance del sistema (ver Arquitectura); este script solo da formato a texto que ya tienes en un archivo local.

1. Coloca el texto crudo en `raw.txt` (un comentario por línea).

2. Ejecuta el script: `python limpiar.py`
3. El script genera `comentarios_x.csv` con columnas `id` , `texto` , `fecha` , `usuario` .
4. Carga el resultado con la opción `1` del menú principal.

Véase también: Referencia de módulos (parámetros exactos de cada función) · Arquitectura del sistema (por qué funciona así el análisis).